TP - n° 1

Université Paris-Saclay

# Algorithmes pour les jeux – Mise en œuvre en Java

Le but de ce TP est de vous familiariser avec les techniques de recherche dans les arbres de jeux. Vous allez écrire un programme JAVA permettant de jouer au jeu des dominos et au jeu de Nim vus en cours.

Vous ferez particulièrement attention à ce que la mise en oeuvre des algorithmes vus en cours reste *générique*. Les algorithmes que vous allez écrire doivent pouvoir en fait fonctionner pour n'importe quel jeu, pourvu que sa mise en oeuvre respecte les interfaces prévues par vos algorithmes. Ceci implique notamment que le code de vos algorithmes ne doit tenir compte d'aucune particularité du jeu des dominos mais uniquement s'appuyer sur les interfaces prévues pour la caractérisation d'un jeu.

Téléchargez l'archive mise en ligne à eCampus.

## I) Organisation du code

L'archive vous est livrée sous la forme d'un projet Gradle (`https://gradle.org`).

Gradle est un utilitaire qui permet d'automatiser un grand nombre de tâches lors d'un développement d'un projet (un peu comme Maven ou Ant). Il fait l'hypothèse que l'organisation du répertoire d'un projet suit certaines conventions, ce qui permet de simplifier un grand nombre de tâches mais suppose aussi **de ne pas changer l'organisation du projet**.

L'archive qui vous a été donnée contient deux entités :

- deux fichiers `build.gradle` et `gradle.setting`. Le premier décrit décrit toutes les caractéristiques du projet gradle et les tâches associées. Ce fichier peut être enrichi pour rajouter des éléments permettant de concevoir et enchainer des taches particulières (mais ce n'est pas nécessaire à votre niveau).

  Le second contient juste un indication sur la racine du projet dans le source.
- un répertoire `src` qui contient le source du projet.

### a) Initialisation du projet

- Déplacez vous dans le répertoire et tapez la commande :

  ```
  gradle build
  ```

  Cette commande a pour effet de lancer la commande permettant de compiler, lancer les test, .... Les fichiers créés par le build seront placés dans un répertoire `build`.

  (Remarque : la toute première utilisation de Gradle, est un peu plus longue que les suivantes, car un certain nombre d'outils sont téléchargés automatiquement pour que tout se passe bien par la suite. Sont notamment créés, un répertoire `gradle`, et deux fichiers `gradlew` et `gradlew.bat`).
- Si vous avez l'habitude d'utiliser l'IDE Eclipse pour développer, taper en plus la commande :

  ```
  gradle :eclipse
  ```

  Cette commande a pour effet de modifier le fichier `build.gradle` pour spécifier qu'il faut utiliser un plugin, dont l'effet sera de générer automatiquement tous les fichiers nécessaires pour la gestion d'un projet sous l'IDE Eclipse .

  Vous pouvez à présent importer votre projet dans un workspace Eclipse en utilisant *Importer un projet Gradle existant*.

  NB : Si vous utilisez un autre IDE, consultez la documentation de votre IDE (ou celle de gradle) pour savoir si un autre plugin est nécessaire.

### b) Contenu du répertoire `src`

Le répertoire `src` contient lui même deux répertoires `main` et `test`. Le premier contient le code principal du projet, le second les fichiers permettant de réaliser des test unitaires (ici il n'y en a pas... mais vous pouvez en rajouter, c'est une bonne pratique à suivre !).

Le code qui vous intéresse se trouve dans le répertoire `src/main/java`. Il contient :

- un package `iialib`, qui regroupe différentes classes liées à ce cours d'*Introduction à l'IA*, lui même organisé en fonction des thématiques abordées. Il contient notamment un package `games`, lui même décomposé en :
  - un package `model`, regroupant l'essentiel des classes servant à la modélisation abstraite de jeux à deux joueurs (ou plus)
  - un packgage `algs` où se trouvent le code lié aux algorithmes de jeu.
- un package `games`, destiné à recevoir les modèles des différents jeux que l'on souhaite coder.

Afin de pouvoir décrire des algorithmes *génériques* sans avoir à faire de suppositions sur la nature des jeux réellement codés, un certain nombre d'interfaces sont proposées, qui modélisent les opérations essentielles qui doivent être implémentées pour pouvoir être traitée par des algorithmes. Par exemple :

- L'interface `IBoard.java` permet de parler d'un plateau d'un jeu, sans avoir à faire trop de supposition sur les caractéristiques des jeux réellement traités. Seule un ensemble restreint de méthodes abstraites sont spécifiées.
- L'interface `IMove.java` permet de représenter un coup lors d'une partie.
- L'interface `IRole.java` permet de représenter distinguer les rôles des joueurs dans un jeu .
- ...

Pour modéliser un jeu réel il faut introduire des classes et des méthodes concrètes pour chacune de ces notions (plateau de jeu, coup de jeu, heuristiques,...).

Dans le code qui vous est donné, vous avez un exemple concret avec le jeu des dominos vu en cours. Vous trouverez notamment une classe `DominosGame` qui montre comment utiliser le tout pour faire s'affronter deux joueurs artificiels utilisant un algorithme de recherche de type minimax.

## Questions sur le jeu des dominos

1. Dans un premier temps, en observant la façon dont le jeu des dominos est modélisé, essayer de comprendre l'articulation des différents composants de cette modélisation.
2. La modélisation du jeu des dominos n'est pas encore tout à fait complète. Il manque en effet, les heuristiques d'évaluation. Compléter le fichier `DominosHeuristics.java` en ce sens, sachant que vous trouverez tout ce qui peut vous être utile pour la coder dans la classe `DominosBoard.java`.

## Algorithmes de jeu

1. Complétez la classe `Minimax` donnée pour coder les algorithmes simples de minimax. Ajoutez à cette classe une technique permettant de compter le nombre de feuilles et de noeuds développés par la recherche.
2. Testez votre minimax contre vous-même sur une grille 4x4 puis sur une grille 6x6, en faisant varier la profondeur de recherche de votre algorithme.

   Si vous avez compilé votre projet avec gradle build vous pouvez lancer la partie à l'aide de la commande :

   ```
   java -cp build/libs/iialib.jar games.dominos.DominosGame
   ```
3. Créez la classe `AlphaBeta` implantant l'algorithme vu en cours (dans sa version Classique et/ou Négamax). Faites jouer votre Alpha-Beta contre votre Minimax en ajustant leurs profondeurs de recherche respectives. Comparez les résultats.

## Questions sur le jeu de Nim

On s'intéresse ici à la modélisation d'une des nombreuses variantes du jeu de Nim, dans laquelle on dispose au départ d'un ensemble de $N$ allumettes et où chaque joueur peut tour à tour enlever $1, 2$ ou 3 allumettes. Le joueur perdant est celui qui ramasse la dernière allumette.

1. Développez l'arbre de recherche entier pour $N = 7$.
2. Existe-t-il des stratégies gagnantes pour ce jeu ?
3. En prenant comme heuristique $h(n) = N - n$ pour $n > 1$, $h(1) = -\infty$ et $h(0) = +\infty$ pour Ami, et $h(1) = +\infty$ et $h(0) = -\infty$ pour Ennemi, simulez une partie entière pour $N = 10$ entre deux I.A., l'une voyant à 2 demi-coups, et l'autre à 4 demi-coups.